Executive Summary

단일 GPU 기반 Multi-Model AI 추론 플랫폼 구축 및 최적화

CJ 4DPLEX AI Platform 개발 사전과제

안하민 | 2026.11.12

1. 프로젝트 개요

본 프로젝트는 **하나의 GPU에서 생성형 AI와 컴퓨터 비전 AI를 동시에 운영할 때 발생하는 성능 저하를 분석하고, 제한된 컴퓨팅 자원을 효율적으로 배분하는 AI 추론 플랫폼을 구현**하는 것을 목표로 한다.

콘텐츠 제작 및 분석 환경에서는 텍스트를 생성하는 AI와 이미지·영상에서 객체를 검출하는 AI가 함께 활용될 수 있다. 그러나 여러 AI 모델이 동일한 GPU를 사용하면 자원 경합으로 인해 처리시간이 증가하고, 대기 중인 요청의 응답까지 지연될 수 있다.

이를 해결하기 위해 단일 NVIDIA DGX Spark 환경에 두 AI 모델을 독립적으로 배포하고, 실제 요청 처리시간과 GPU 자원 사용 특성을 분석하였다. 이후 CUDA MPS 기반의 GPU 자원 분할을 적용하여 모델 간 간섭을 줄이고, 전체 서비스 성능을 개선하는 방법을 검증하였다.

2. 시스템 구성 및 최적화 접근

본 플랫폼은 다음 두 모델로 구성된다.

- **Qwen3.5-9B:** 사용자 입력에 대한 텍스트 생성
- **YOLO26l:** 이미지 내 객체 검출

각 모델은 Kubernetes 기반의 독립적인 Pod에서 실행되며, ROS 2를 통해 요청을 수신한다. 접수된 요청은 모델별 FIFO Queue에 저장되고, 전용 Worker가 순차적으로 처리한다. 두 모델은 서로 독립적으로 동작하지만 하나의 물리 GPU를 공유한다.

성능 분석 과정에서 두 모델의 GPU 자원 특성에 차이가 있음을 확인하였다.

Qwen은 입력을 분석하는 초기 단계에서 GPU 연산 자원의 영향을 크게 받았으나, 이후 문장을 생성하는 반복 단계에서는 자원 증가에 따른 처리시간 개선이 상대적으로 작았다. 반면 YOLO는 GPU 연산 자원을 추가로 사용할수록 이미지 처리시간이 뚜렷하게 감소하였다.

이러한 특성을 바탕으로 **Qwen에 필요한 자원을 제한적으로 배분하고, 나머지 자원을 YOLO에 집중하는 비대칭 자원 분할 전략**을 적용하였다.

GPU의 총 48개 SM(연산 자원 단위)을 여러 비율로 분할하여 비교한 결과, 평가한 조건에서는 **Qwen 8SM / YOLO 40SM** 구성이 가장 유리한 전체 작업 완료시간과 YOLO 처리 성능을 나타냈다.

3. 주요 성과

최신 실험에서는 Qwen 요청 10건과 YOLO 이미지 처리 요청 300건을 동시에 발행하고, 동일한 작업량에서 GPU 자원 배분 방식에 따른 성능을 비교하였다. 각 설정은 5회 반복 측정하였다.

핵심 성능 비교

평가 지표	최적화 전 (MPS 없음)	최적화 후 (8/40SM)	결과
YOLO 300건 완료시간	16.322초	**11.592초**	**28.98% 단축**
YOLO 요청 E2E p95	14.963초	**10.674초**	**28.66% 단축**
전체 310건 완료시간	113.361초	**111.887초**	1.30% 단축

요청 E2E는 대기시간을 포함한 실제 요청·응답 구간이며, p95는 측정된 요청 지연 분포의 95번째 백분위수이다. 본 결과는 정해진 요청을 한 번에 발행한 Burst 실험에 한정된다.

추가로, 단순히 MPS를 활성화한 일반 GPU 공유 방식과 비교하면 YOLO 300건의 완료시간을 **54.31% 단축**하였다.

이는 두 모델을 단순히 병렬 실행하는 것보다, 각 모델의 처리 특성에 맞춰 GPU 자원을 분배하는 것이 효과적일 수 있음을 보여준다.

이번 최적화에서 중요한 점은 **YOLO의 처리시간을 줄이면서도 동일한 실험 조건에서 Qwen의 전체 응답 완료시간을 크게 악화시키지 않았다는 것**이다.

또한 처리시간 감소는 후속 요청의 Queue 대기시간 감소로 이어졌다. 요청을 삭제하거나 작업량을 줄이지 않고 동일한 입력과 모델을 유지한 상태에서 얻은 개선이다.

4. 운영 안정성 및 장애 대응

성능 최적화와 함께 플랫폼 운영 과정에서 발생할 수 있는 장애도 조사하였다.

실제 운영 중 Kubernetes Pod가 Ready 상태임에도 ROS 2 요청이 전달되지 않는 문제가 발생하였다. 분석 결과 FastDDS의 공유 메모리 자원과 사용자 세션 종료 시 IPC 정리 정책의 연관성을 확인하였으며, 계정 및 Pod 설정을 조정한 뒤 요청이 정상적으로 처리되는 것을 검증하였다.

또한 YOLO 프로세스를 의도적으로 종료하는 장애 실험을 수행하였다.

유휴 상태에서 프로세스를 강제 종료한 3회 실험에서 Kubernetes의 기본 재시작 기능을 통해 **평균 4.39초 만에 첫 정상 응답을 다시 수신**하였다.

이를 통해 기본적인 프로세스 장애 복구 동작을 확인하였다. 다만 해당 결과는 유휴 상태에서의 장애 실험으로, 실제 추론 중 장애에 대한 무손실 처리나 무중단 서비스를 보장하는 것은 아니다.

5. 주요 리스크 및 한계

이번 실험은 제한된 환경에서 자원 분할 전략의 효과를 검증한 것으로, 실제 서비스에 적용하기 위해서는 다음 사항을 추가로 고려해야 한다.

첫째, AI 모델 간 성능 Trade-off가 존재한다.

Qwen의 전체 작업 완료시간은 유지되었지만, 내부 첫 토큰 생성시간은 평균 약 188ms에서 472ms로 증가하였다. 따라서 사용자에게 첫 응답을 빠르게 제공해야 하는 대화형 서비스에서는 다른 자원 배분이 더 적합할 수 있다.

둘째, 현재 자원 배분은 고정되어 있다.

YOLO 요청이 없더라도 Qwen이 YOLO에 할당된 SM을 자동으로 활용하지는 못한다. 요청량이 시간에 따라 변하는 실제 서비스에서는 유휴 자원 문제가 발생할 수 있다.

셋째, 실험 부하의 대표성이 제한적이다.

동일한 입력 길이와 이미지로 구성된 일괄 요청을 평가했으며, 지속적으로 요청이 유입되는 환경의 지연, 장기 가용성, 다양한 입력에 대한 성능은 아직 검증하지 않았다.

따라서 이번 결과는 특정 실험 조건에서의 최적화 성과이며, 실제 운영 환경의 SLO 달성을 의미하지는 않는다.

6. 향후 개선 방향

실제 서비스 운영을 위한 후속 과제로 다음 세 가지를 제안한다.

1. **서비스 특성에 따른 자원 배분 정책 고도화**

대화형 AI의 응답속도와 이미지 처리량 중 우선순위가 높은 지표에 따라 GPU 자원 배분을 선택하고, 요청량 변화에 따른 동적 재배분의 필요성을 평가한다.

2. **전체 추론 파이프라인의 추가 최적화**

GPU 모델 연산뿐 아니라 요청 전달, Queue 대기, 결과 반환 과정의 지연을 세분화하여 분석하고, 모델 외 처리 구간의 병목을 줄인다.

3. **운영 안정성 및 서비스 수준 검증**

지속 부하 실험, Queue 포화 대응, 요청 제한 및 재시도 정책, 추론 중 장애 복구 등을 검증하고 실제 업무 요구사항에 기반한 SLO를 수립한다.

7. 결론

본 프로젝트는 **AI 모델별 연산 특성에 기반한 GPU 자원 배분이 단일 GPU의 Multi-Model 추론 성능을 개선할 수 있음을 실험적으로 확인**하였다.

모델별 독립적인 운영 구조를 구축하고, 성능 저하 원인을 분석한 후 CUDA MPS 정적 분할을 적용한 결과, Qwen의 전체 응답 성능을 유지하면서 YOLO의 일괄 처리 완료시간을 약 29% 단축하였다.

또한 성능 개선에 따른 비용과 장애 복구의 검증 범위를 함께 분석함으로써, **제한된 컴퓨팅 자원에서 서로 다른 AI 서비스를 안정적이고 효율적으로 운영하기 위한 기술적 기반**을 마련하였다.